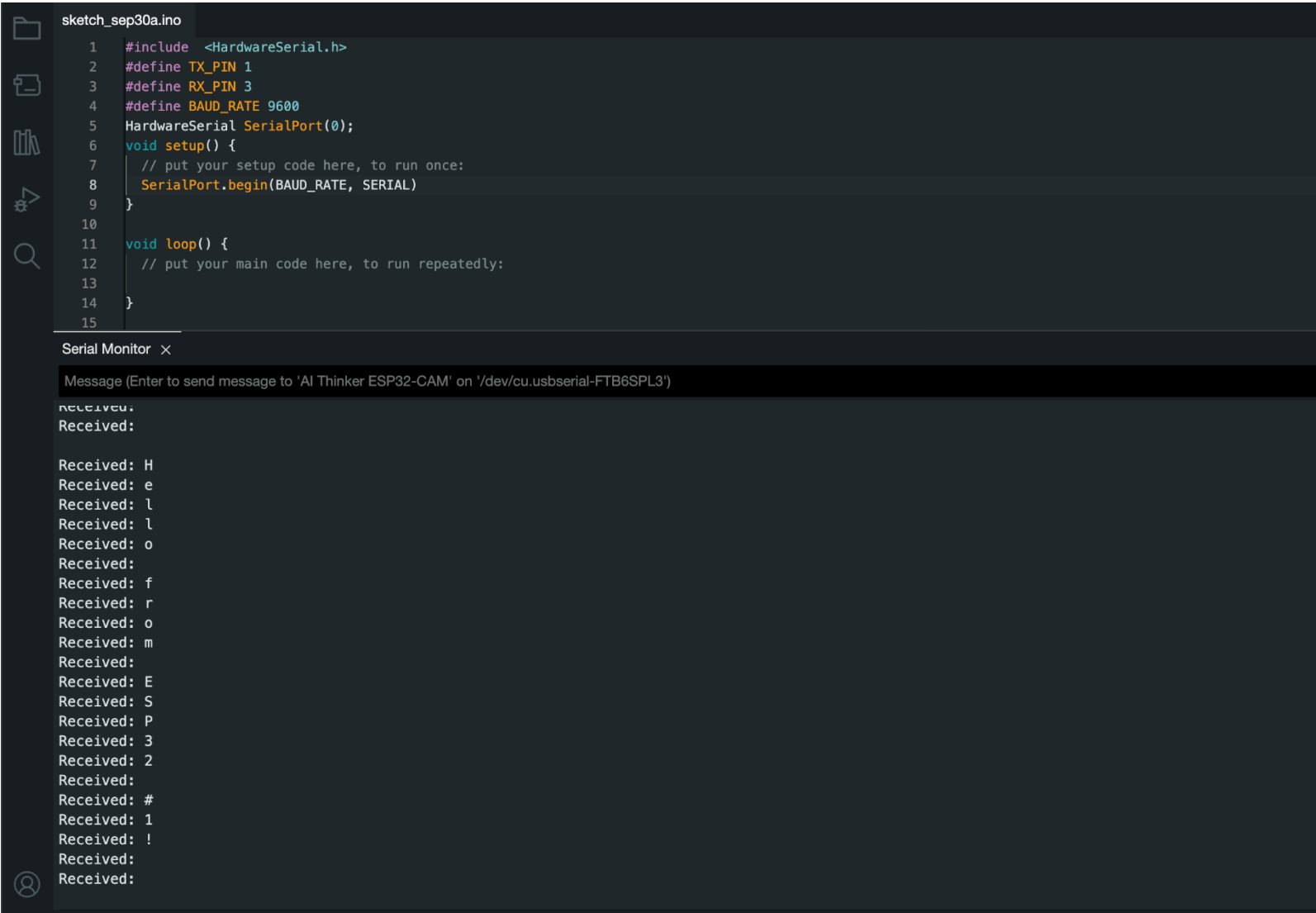


EE 4370 Lab 6

Part 1

For this lab, we are using UART communication which is known as wired connection communication. In our program we used Arduino functions to send messages between the two microcontrollers. Provided with the code from part 1 our output is listed below, which matches with the expected result from our code.

The image is a screenshot of an IDE, likely Arduino IDE, showing a sketch and its serial monitor output. The sketch is named 'sketch_sep30a.ino' and contains 15 lines of code. The code defines TX_PIN as 1, RX_PIN as 3, and BAUD_RATE as 9600. It initializes a HardwareSerial object 'SerialPort' at pin 0. The setup function calls SerialPort.begin(BAUD_RATE, SERIAL). The loop function is currently empty. The serial monitor shows the output of the program, which is a series of characters: 'Received:', 'Received:', 'Received: H', 'Received: e', 'Received: l', 'Received: l', 'Received: o', 'Received:', 'Received: f', 'Received: r', 'Received: o', 'Received: m', 'Received:', 'Received: E', 'Received: S', 'Received: P', 'Received: 3', 'Received: 2', 'Received:', 'Received: #', 'Received: 1', 'Received: !', 'Received:', and 'Received:'. The serial monitor has a text input field at the top with the placeholder text 'Message (Enter to send message to 'AI Thinker ESP32-CAM' on '/dev/cu.usbserial-FTB6SPL3')'.

Part 2

In this part we want to replicate a similar outcome from part 1. In this implementation we are using only Espressif functions which are specific to the ESP 32. Using these over Arduino code gives us the ability to modify additional parameters meaning we have more options for what we want the program to do. In the output we got a message which was transmitted to the receiver just like part 1. However, the format of how it was printed in part 1 and 2 is different. In part 2 the message a fit in one line and looked like normal text. While in part 1 the message included one character at the end of each line to make the message. The outcome from part 1 and 2 is different, but we know that our program worked correctly.

[illegible]

Part 3

In the last part of the lab, we revised our code from part 2 to implement switch input from the transmitter to the receiver which would have a led output. The case was if switch 1 was pressed then green led would be on and if switch 2 is pressed red led is on. The code is the same from the void setup section. I mainly changed the loop code as that is where we would transmit and receive the bits whether it is green or red. As shown in the transmitter code we still use const char data to store our input from the switch then that is sent over to the receiver code. In the receiver's code the char is read and based on the value a led is turned on.

EE_4370_Lab_Gold

```
// Part 3
#include "driver/uart.h"
#define TX_PIN 1
#define RX_PIN 3
// Define pin numbers to variables
const int switch1 = 16;
const int switch2 = 15;
void setup() {
    Serial.begin(9600); // Set baud rate
    // Set configuration
    uart_config_t uart_config = {
        .baud_rate = 9600,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE
    };
    uart_param_config(UART_NUM_0, &uart_config);
    uart_set_pin(UART_NUM_0, TX_PIN, RX_PIN, UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);
    uart_driver_install(UART_NUM_0, 1024, 0, 0, NULL, 0);
}
void loop() {
    const char* data; // Declare a char variable named data
    // Send character 'g' to receiver if switch 1 is pressed
    if (digitalRead(switch1)) {
        data = "g";
        uart_write_bytes(UART_NUM_0, data, strlen(data));
        delay(1000);
    }
    // Send character 'r' to receiver if switch 2 is pressed
    if (digitalRead(switch2)) {
        data = "r";
        uart_write_bytes(UART_NUM_0, data, strlen(data));
        delay(1000);
    }
}
```

Lab6.ino

```
1  #include "driver/uart.h"
2
3  #define TX_PIN 1
4  #define RX_PIN 3
5  #define LED_PIN_1 14
6  #define LED_PIN_2 15
7
8  void setup() {
9      Serial.begin(9600);
10
11     uart_config_t uart_config = {
12         .baud_rate = 9600,
13         .data_bits = UART_DATA_8_BITS,
14         .parity = UART_PARITY_DISABLE,
15         .stop_bits = UART_STOP_BITS_1,
16         .flow_ctrl = UART_HW_FLOWCTRL_DISABLE
17     };
18
19     uart_param_config(UART_NUM_0, &uart_config);
20     uart_set_pin(UART_NUM_0, TX_PIN, RX_PIN, UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);
21     uart_driver_install(UART_NUM_0, 1024, 0, 0, NULL, 0);
22
23     pinMode(LED_PIN_1, OUTPUT);
24     pinMode(LED_PIN_2, OUTPUT);
25 }
26
27 void loop() {
28     uint8_t* data = (uint8_t*) malloc(1024);
29     int len = uart_read_bytes(UART_NUM_0, data, 1024, 20 / portTICK_RATE_MS);
30
31     if (len > 0) {
32         data[len] = '\0';
33         if (data[0] == 'g') {
34             digitalWrite(LED_PIN_1, HIGH);
35             digitalWrite(LED_PIN_2, LOW);
36
37         } else if (data[0] == 'r') {
38             digitalWrite(LED_PIN_1, LOW);
39             digitalWrite(LED_PIN_2, HIGH);
40
41         } else {
42             digitalWrite(LED_PIN_1, LOW);
43             digitalWrite(LED_PIN_2, LOW);
44         }
45     } else if (len < 0) {
46         Serial.println("UART read error occurred.");
47     }
48
49     free(data);
50 }
51
```